



Universidad Autónoma de Baja California

Programación Orientada a Objetos

Programación Orientada a Objetos

Constructores, sobrecarga de métodos

Profesor: MC. Itzel Barriba Cázares

Pase de parametros

//This program demonstrates a method with a parameter.

```
public class PaseParametros {  
    public static void main(String[] args) {  
        int x = 10;  
  
        System.out.println("Pasando valores al metodo mostrarValor.");  
        mostrarValor(5);  
        mostrarValor(x);  
        mostrarValor(x * 4);  
        mostrarValor(Integer.parseInt("700"));  
        System.out.println("De nuevo en el main.");  
    }  
  
    // El metodo mostrarValor muestra el valor de la variable entera num.  
    public static void mostrarValor(int num) {  
        System.out.println("El valor es " + num);  
    }  
}
```

Salida del código

```
Pasando valores al metodo mostrarValor.  
El valor es 5  
El valor es 10  
El valor es 40  
El valor es 700  
De nuevo en el main.
```

Llamar un método dentro de un ciclo

```
/**
 * Este programa define y llama un método.
 */

public class LlamarMetodoCiclo {
    public static void main(String[] args) {
        System.out.println("Hola desde el metodo main.");
        for (int i = 0; i < 5; i++)
            mostrarMensaje();
        System.out.println("De nuevo en el metodo main.");
    }

    /**
     * El metodo mostrarMensajes despliega un saludo.
     */

    public static void mostrarMensaje() {
        System.out.println("Hola desde el metodo mostrarMensaje.");
    }
}
```

Salida del código

```
Hola desde el metodo main.
Hola desde el metodo mostrarMensaje.
Hola desde el metodo mostrarMensaje.
Hola desde el metodo mostrarMensaje.
Hola desde el metodo mostrarMensaje.
Hola desde el metodo mostrarMensaje.
De nuevo en el metodo main.
```


Llamadas a metodos de forma jerárquicos

- Los métodos también se pueden llamar de forma jerárquica o por capas. Es decir, un métodoA puede llamar al métodoB que a su vez llama a un métodoC.

//Programa que demuestra llamada de metodos jerarquicos.

```
public class LlamadaMetodosJerarquicos {
    public static void main(String[] args) {
        System.out.println("Iniciamos en el metodo main.");
        deep();
        System.out.println("Ahora estoy de nuevo en el main.");
    }

    // El metodo main muestra un mensaje y despues llama al metodo deeper.

    public static void deep() {
        System.out.println("Estoy en en metodo deep.");
        deeper();
        System.out.println("Ahora de nuevo estoy en el metodo deep.");
    }

    //El metodo deeper simplemente muestra un mensaje.

    public static void deeper() {
        System.out.println("Ahora estoy en deeper.");
    }
}
```

Variables locales

- ▶ Una **variable local** se declara dentro de un método y no es accesible para instrucciones fuera del método.
- ▶ Diferentes métodos pueden tener variables locales con el mismo nombre porque los métodos no pueden ver las variables de los demás.

Variables locales

//Metodo que muestra que dos metodos tienen variables locales con el mismo nombre

```
public class VarLocales {
    public static void main(String[] args) {

        texas();
        california();
    }

    /**
     * Metodo texas tiene una variable local llamado pajaros
     */
    public static void texas() {
        int pajaros = 5000;
        System.out.println("En texas hay " + pajaros + " pajaros.");
    }

    /**
     * Metodo california tiene tambien una variable local llamado pajaros
     */
    public static void california() {
        int pajaros = 3500;
        System.out.println("En california hay " + pajaros + " pajaros.");
    }
}
```


Duración de una variable

- ▶ Las variables locales:
 - ▶ Se crean cuando inicia el métodos (Cuando el método comienza, sus variables locales y sus variables de parámetros se crean en la memoria)
 - ▶ Se destruyen cuando termina
 - ▶ No conservan su valor entre llamadas.

Sobrecarga de Métodos (overloading)

- ▶ La sobrecarga ocurre cuando:
 - ▶ Existen varios métodos con el mismo nombre pero diferentes parámetros.
- ▶ Se diferencias por:
 - ▶ según el número de parámetros
 - ▶ Tipos de parametros
 - ▶ Orden de los parámetros.
- ▶ Esto aplica también para constructores

Sobrecarga de Métodos (overloading)

```
public class Book {  
    String titulo;  
    String autor;  
    boolean esLeido;  
    int numberDeLecturas;  
  
    public void leer(){  
        esLeido = true;  
    }  
    public void leer(){  
        numberDeLecturas++;  
    }  
}
```

- Si prueba este ejemplo, verá que hay errores que indican que el método `leer()` está duplicado. El problema es que tienes dos métodos con el mismo nombre y ninguno tiene parámetros.

Sobrecarga de Métodos (overloading)

```
public class Book {  
    String titulo;  
    String autor;  
    boolean esLeido;  
    int numberDeLecturas;  
  
    public void leer(){  
        esLeido = true;  
        numberOfReadings++;  
    }  
    public void leer(int i){  
        isRead = true;  
        numberOfReadings += i;  
    }  
}
```

- Ahora tiene un método con el nombre `leer()` y sin parámetros y otro método con el nombre `leer()` y un parámetro `int`. Para Java, estos son dos métodos completamente diferentes, por lo que no tendrás errores de duplicación.

Ejemplo: sobrecarga de métodos

```
// Sobre carga de métodos.  
public class SobreCargaMetodos {  
  
    public static int cuadrado(int num) {  
        System.out.printf("%nLlamada al metodo 1 con int: %d%n", num);  
        return num * num;  
    }  
  
    public static double cuadrado(double num) {  
        System.out.printf("%nLlamada al método 2 con doble : %f%n", num);  
        return num * num;  
    }  
  
    public static void main(String[] args) {  
        System.out.printf("Cuadrado del entero 7 es %d%n", cuadrado(7));  
        System.out.printf("Cuadrado del entero 7.5 es %f%n", cuadrado(7.5));  
    }  
}
```

Salida del código

```
Llamada al metodo 1 con int: 7  
Cuadrado del entero 7 es 49  
  
Llamada al método 2 con doble : 7.500000  
Cuadrado del entero 7.5 es 56.250000
```


Ejemplo: sobrecarga de constructores

```
public class Caja8 {
    double ancho;
    double altura;
    double profundidad;

    //sobrecarga de constructores
    // Este es el constructor de Caja que inicializa con valores definidos.
    Caja8() {
        ancho = 10;
        altura = 20;
        profundidad = 30;
    }

    // Este es el constructor de Caja que recibe tres parametros.
    Caja8(double ancho, double altura, double profundidad) {
        this.ancho = ancho;
        this.altura = altura;
        this.profundidad = profundidad;
    }

    // Este es el constructor de Caja cuadrada recibe un parametro.
    Caja8(double a) {
        ancho = a;
        altura = a;
        profundidad = a;
    }

    //metodos getters
    public double getAncho() {
        return ancho;
    }

    public double getAltura() {
        return altura;
    }

    public double getProfundidad() {
        return profundidad;
    }

    // calcula y regresa el volumen
    double volumen() {
        return ancho * altura * profundidad;
    }
}
```

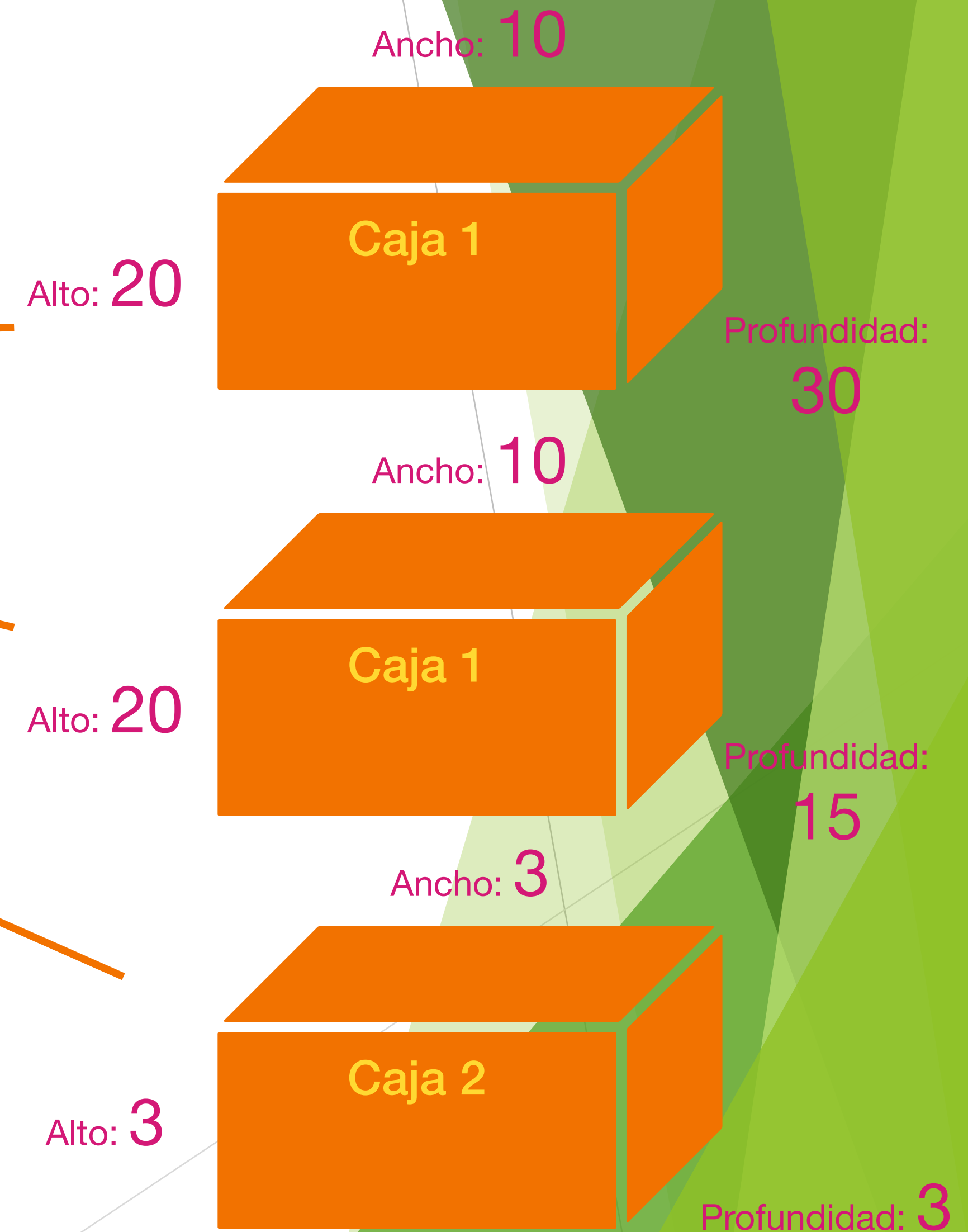
Caja
- ancho: double - altura: double - profundidad: double
+ Caja(): + Caja(ancho: double, altura: double, profundidad: double): + Caja(a: double): + getAncho(): double + getAltura(): double + getProfundidad(): double + volumen(): double

Prueba de Constructor con parámetros

```
public class DemoCaja8 {  
    public static void main(String[] args) {  
        // declara, aloca e inicializa objetos Caja  
        Caja8 miCaja1 = new Caja8();  
        Caja8 miCaja2 = new Caja8(10,20,15);  
        Caja8 miCaja3 = new Caja8(3);  
        double vol;  
  
        vol = miCaja1.volumen();  
        System.out.println("Las dimensiones de la caja son: ");  
        System.out.println("ancho: "+miCaja1.getAncho());  
        System.out.println("altura: "+miCaja1.getAltura());  
        System.out.println("profundidad: "+miCaja1.getProfundidad());  
        System.out.println("Volumen es " + vol);  
  
        vol = miCaja2.volumen();  
        System.out.println("Las dimensiones de la caja son: ");  
        System.out.println("ancho: "+miCaja2.getAncho());  
        System.out.println("altura: "+miCaja2.getAltura());  
        System.out.println("profundidad: "+miCaja2.getProfundidad());  
        System.out.println("Volumen es " + vol);  
  
        vol = miCaja3.volumen();  
        System.out.println("Las dimensiones de la caja son: ");  
        System.out.println("ancho: "+miCaja3.getAncho());  
        System.out.println("altura: "+miCaja3.getAltura());  
        System.out.println("profundidad: "+miCaja3.getProfundidad());  
        System.out.println("Volumen es " + vol);  
    }  
}
```

Salida del código

```
Las dimensiones de la caja 1 son:  
ancho: 10.0  
altura: 20.0  
profundidad: 30.0  
Volumen es 6000.0  
Las dimensiones de la caja 2 son:  
ancho: 10.0  
altura: 20.0  
profundidad: 15.0  
Volumen es 6000.0  
Las dimensiones de la caja 3 son:  
ancho: 3.0  
altura: 3.0  
profundidad: 3.0  
Volumen es 27.0
```



Prueba de Constructor con parámetros y captura de datos

```
public class DemoCaja8 {
    public static void main(String[] args) {
        double ancho, altura, profundidad, vol;
        Scanner input = new Scanner(System.in);
        System.out.println("Ingrese el ancho de la caja");
        ancho = input.nextDouble();
        System.out.println("Ingrese el altura de la caja");
        altura = input.nextDouble();
        System.out.println("Ingrese el profundidad de la caja");
        profundidad = input.nextDouble();

        // declara, aloca e inicializa objetos Caja
        Caja8 miCaja1 = new Caja8();
        Caja8 miCaja2 = new Caja8(ancho, altura, profundidad);
        Caja8 miCaja3 = new Caja8(3);
        double vol;

        vol = miCaja1.volumen();
        System.out.println("Las dimensiones de la caja son: ");
        System.out.println("ancho: "+miCaja1.getAncho());
        System.out.println("altura: "+miCaja1.getAltura());
        System.out.println("profundidad: "+miCaja1.getProfundidad());
        System.out.println("Volume is " + vol);

        vol = miCaja2.volumen();
        System.out.println("Las dimensiones de la caja son: ");
        System.out.println("ancho: "+miCaja2.getAncho());
        System.out.println("altura: "+miCaja2.getAltura());
        System.out.println("profundidad: "+miCaja2.getProfundidad());
        System.out.println("Volume is " + vol);

        vol = miCaja3.volumen();
        System.out.println("Las dimensiones de la caja son: ");
        System.out.println("ancho: "+miCaja3.getAncho());
        System.out.println("altura: "+miCaja3.getAltura());
        System.out.println("profundidad: "+miCaja3.getProfundidad());
        System.out.println("Volume is " + vol);
    }
}
```

Salida del código

```
Las dimensiones de la caja 1 son:
ancho: 10.0
altura: 20.0
profundidad: 30.0
Volumen es 6000.0
Las dimensiones de la caja 2 son:
ancho: 10.0
altura: 20.0
profundidad: 15.0
Volumen es 6000.0
Las dimensiones de la caja 3 son:
ancho: 3.0
altura: 3.0
profundidad: 3.0
Volumen es 27.0
```



Resumen

- ▶ El nombre del constructor tienen el mismo nombre de la clase.
- ▶ Una clase puede tener más de un constructor
- ▶ Este constructor se ejecuta cuando se construyen objetos.
- ▶ Un constructor puede tomar cero, uno o más parámetros
- ▶ Un constructor no tiene valor de retorno.
- ▶ Un constructor siempre se llama con el operador new